

RIPASSO TECNICO

Angular Router

Lazy loading, guard, preloading

Bundle splitting, la famiglia completa dei guard, strategie di preloading — tre esperimenti dal vivo, verificati via rete e via CDP

Costruito a partire da una conversazione con Claude

Progetto di riferimento: `angular-vue-crash-project` (entrambi i branch)

Agosto 2026

Indice

1. Il router come pipeline di decisioni
2. Lazy loading: `loadComponent`
3. Demo dal vivo 1: bundle splitting verificato via build e via rete
4. Strategie di preloading
5. Demo dal vivo 2: la differenza di timing con/senza preloading
6. La famiglia dei guard: `CanActivate`, `CanDeactivate`, `CanActivateChild`, `CanMatch`
7. Demo dal vivo 3: `CanDeactivate` su entrambi i branch (bloccato, permesso, nessun falso positivo)
8. Resolver: cosa sono, e perché questo progetto non ne ha bisogno
9. Rotte annidate, in breve
10. Cheat sheet finale
11. Appendice: codice sorgente completo

1. Il router come pipeline di decisioni

Il modello mentale utile per il router di Angular: ogni navigazione attraversa una **pipeline di decisioni**, in ordine — prima "questo pezzo di codice va scaricato?" (`CanMatch`), poi "posso entrare in questa rotta?" (`CanActivate` / `CanActivateChild`), poi eventualmente "che dati servono prima di mostrare il componente?" (resolver), e quando si naviga *via* da una rotta, "posso davvero uscire?" (`CanDeactivate`). Il progetto Task Board usava finora solo il primo pezzo di questa pipeline (`CanActivate` , per proteggere `/new`); questo manuale copre il resto, implementato e verificato dal vivo in questa sessione.

CONFRONTO CON VUE ROUTER

Vue Router ha lo stesso concetto di pipeline: `beforeEnter` / `beforeEach` ($\approx$ `CanActivate`), `beforeRouteLeave` ($\approx$ `CanDeactivate`), lazy loading via `() => import('./View.vue')` nella config della rotta (praticamente identico a `loadComponent`). La differenza principale è che Angular separa questi concetti in funzioni/tipi dedicati e dichiarativi nella config della rotta, mentre Vue Router li espone più spesso come hook/metodi sul componente o callback globali.

2. Lazy loading: `loadComponent`

Prima di questa sessione, ogni rotta del progetto usava `component: TaskForm` — il codice di **ogni** feature finiva nel bundle iniziale, scaricato tutto insieme all'avvio dell'app, anche per pagine che l'utente potrebbe non visitare mai in quella sessione. Con `loadComponent`, il codice di quella singola feature diventa un **chunk JavaScript separato**, scaricato solo quando serve davvero:

```
// prima
{ path: 'new', component: TaskForm, canActivate: [canActivateNewTask] }

// dopo
{
  path: 'new',
  loadComponent: () => import('./features/task-form/task-form').then((m) =>
m.TaskForm),
  canActivate: [canActivateNewTask],
  canDeactivate: [canDeactivateTaskForm],
}
```

Nel progetto, tutte le rotte feature usano ora `loadComponent` tranne quella di default (`TaskList`, la home) — resta "eager" perché si carica comunque all'avvio, separarla in un chunk a parte non avrebbe alcun beneficio.

3. Demo dal vivo 1: bundle splitting verificato via build e via rete

VERIFICA 1 — OUTPUT DI NG BUILD

Initial chunk files	Names	Raw size	Estimated transfer size
main-GQIF0HLZ.js	main	286.00 kB	77.88 kB
styles-MEH0MT6X.css	styles	13.62 kB	3.06 kB
Lazy chunk files	Names	Raw size	Estimated transfer size
chunk-8lv5QBda.js	task-form	60.79 kB	15.41 kB
chunk-DIrre3ky.js	task-search	4.15 kB	1.87 kB
chunk-UST8K1ln.js	task-detail	1.99 kB	896 bytes
chunk-BkxXn7DQ.js	login	1.46 kB	683 bytes

Quattro chunk lazy separati, uno per feature, con nomi leggibili assegnati automaticamente da Angular in base al componente esportato.

VERIFICA 2 — RICHIESTE DI RETE REALI (SCRIPT NODE CON WEBSOCKET/CDP)

```
--- Navigazione iniziale su / ---  
Chunk .js richiesti al primo caricamento:  
[ 'main.js', 'chunk-LNJLHLXB.js', 'chunk-VBS3KEGI.js', 'chunk-VUJ0FXKG.js' ]  
  
--- Ora navigo su /search cliccando il link in nav ---  
Chunk .js richiesti DOPO la navigazione a /search:  
[ 'main.js', 'chunk-LNJLHLXB.js', 'chunk-VBS3KEGI.js', 'chunk-VUJ0FXKG.js',  
  'chunk-INJPEZZQ.js' ]  
  
Chunk NUOVI caricati solo ora: [ 'chunk-INJPEZZQ.js' ] ← esattamente task-  
search, nessun altro
```

Al primo caricamento vengono richiesti solo `main.js` e i chunk "iniziali" condivisi (codice comune, non associato a una feature specifica) — nessuno dei 4 chunk lazy. Solo dopo aver cliccato il link "Cerca (RxJS)" arriva la richiesta per il chunk di `task-search`, e solo quello: `login / task-form / task-detail` restano non richiesti perché non ci si è mai navigato.

4. Strategie di preloading

Il lazy loading da solo ha un costo: il **primo** click su una rotta lazy deve aspettare il download del chunk prima di mostrare qualcosa — un piccolo ma percepibile ritardo che con `component` diretto non esisteva. Le **strategie di preloading** risolvono il compromesso: scaricano i chunk lazy **in background**, subito dopo il boot dell'app, così sono già in cache quando l'utente li richiede davvero.

```
// app.config.ts
provideRouter(
  routes,
  withComponentInputBinding(),
  withPreloading(PreloadAllModules), // <-- nuovo
),
```

`PreloadAllModules` è la strategia built-in più semplice: precarica **tutte** le rotte lazy, indiscriminatamente. Per essere più selettivi (es. precaricare solo alcune rotte in base a una condizione) si può scrivere una strategia custom implementando l'interfaccia `PreloadingStrategy`, tipicamente combinata con la proprietà `data` sulla rotta (es. `{ data: { preload: true } }`) per marcare quali precaricare.

5. Demo dal vivo 2: la differenza di timing con/senza preloading

PRIMA DI WITHPRELOADING (§3, VERIFICA 2)

Solo `task-search` veniva scaricato, e solo **dopo** il click sul link — gli altri tre chunk (`login` , `task-form` , `task-detail`) non venivano mai richiesti finché non ci si navigava.

DOPO WITHPRELOADING(PRELOADALLMODULES)

```
Carico SOLO / e NON navigo da nessuna parte...  
Chunk .js richiesti subito dopo il caricamento (t=500ms):  
[ 'main.js', 'chunk-LNHLHLXB.js', 'chunk-VBS3KEGI.js', 'chunk-VUJ0FXKG.js',  
  'chunk-XBYPCSLP.js', 'chunk-INJPEZZQ.js', 'chunk-PY4Q7DMR.js', 'chunk-  
WDBSW04W.js' ]
```

```
Chunk .js richiesti dopo 2.5s totali, SENZA alcuna navigazione utente:  
[ ...stessi identici otto chunk... ]
```

Entro **500ms** dal solo caricamento di `/` , senza un solo click dell'utente, **tutti e quattro** i chunk lazy (`login` / `task-search` / `task-detail` / `task-form`) sono già stati scaricati in background. Quando l'utente clicca un link, il chunk è già in cache — navigazione percepita come istantanea.

6. La famiglia dei guard

Guard	Decide	Già nel progetto?
CanActivate	Posso ENTRARE in questa rotta?	Sì, da prima (canActivateNewTask , protegge /new)
CanDeactivate	Posso USCIRE da questa rotta?	Nuovo in questa sessione (canDeactivateTaskForm)
CanActivateChild	Come CanActivate, ma per TUTTI i figli di una rotta in un colpo solo	Non usato (nessuna rotta annidata in questo progetto)
CanMatch	Questa rotta esiste nemmeno per questo URL?	Non usato

`CanActivateChild` : utile per proteggere un intero sottoalbero di rotte (es. `/admin/*`) con un guard dichiarato **una sola volta** sul genitore, invece di ripeterlo su ogni figlio.

`CanMatch` (erede del vecchio `CanLoad`) : se ritorna `false`, Angular prova la **prossima** definizione di rotta che combacia con lo stesso path, invece di bloccare/redirect come fa `CanActivate` — utile per feature flag (nascondere l'esistenza stessa di una rotta, non solo negarne l'accesso) o A/B test. Vantaggio in più per le rotte lazy: gira **prima** di scaricare il chunk, quindi se non combacia il codice non viene nemmeno richiesto — un guard e un'ottimizzazione di lazy loading insieme.

CanDeactivate in dettaglio: il nuovo guard di questo progetto

L'opposto di `CanActivate` : non decide se puoi entrare, ma se puoi **uscire**. Caso d'uso implementato: bloccare l'uscita da `/new` se c'è un titolo scritto e non ancora salvato.

```
export const canDeactivateTaskForm: CanDeactivateFn<TaskForm> = (component) => {
  if (!component.hasUnsavedChanges()) {
    return true;
  }
  return window.confirm(
    'Hai scritto un titolo che non hai ancora salvato. Vuoi davvero uscire senza creare il task?',
  );
};
```

DIFFERENZA CHIAVE RISPETTO A CANACTIVATEFN

`CanDeactivateFn` riceve come primo argomento l'**istanza del componente** che si sta per abbandonare — può interrogarlo direttamente (`component.hasUnsavedChanges()`) invece di dover tenere lo stato altrove (es. in un service condiviso solo per questo). Il metodo deve essere `public` sul componente, non `protected`: a differenza dei campi letti solo dal template (dove `protected` basta, vedi la nota su `login.ts` nel manuale principale), qui la chiamata arriva da **fuori la classe**, da un file completamente diverso — `protected` non sarebbe sufficiente.

IL DETTAGLIO CHE SI SCOPRE SOLO IMPLEMENTANDOLO: IL RESET POST-SUBMIT

Senza un accorgimento, il guard bloccherebbe **anche** la navigazione legittima subito dopo un submit riuscito — il modello conterrebbe ancora il titolo appena inviato, letto come "non salvato".
Fix: resettare il modello subito dopo l'aggiunta allo store, prima di navigare:

```
this.store.add(this.model().title);  
this.model.set({ title: '' }); // altrimenti hasUnsavedChanges() resta true  
this.router.navigateByUrl('/');
```


7. Demo dal vivo 3: CanDeactivate su entrambi i branch

Verificato con uno script CDP che simula un utente reale: login, scrittura di un titolo senza invio, tentativo di navigare via, risposta al dialog di conferma (sia annullando che confermando), e infine un submit regolare per verificare l'assenza di falsi positivi.

SCENARIO 1 — TENTATIVO DI USCITA, ANNULLA

```
3. Provo a uscire cliccando "Lista task" (dovrebbe apparire un confirm)...  
[DIALOG APERTO] confirm - Hai scritto un titolo che non hai ancora salvato...  
4. Rispondo ANNULLA al dialog: la navigazione deve restare bloccata...  
   URL dopo ANNULLA: /new ← bloccato correttamente
```

SCENARIO 2 — NUOVO TENTATIVO, STAVOLTA OK

```
5. Riprovo a uscire, stavolta rispondo OK...  
[DIALOG APERTO] confirm - Hai scritto un titolo che non hai ancora salvato...  
   URL dopo OK: / ← navigazione permessa dopo conferma
```

SCENARIO 3 — SUBMIT REGOLARE, NESSUN FALSO POSITIVO

```
6. Torno su "Nuovo task", scrivo un titolo e invio REGOLARMENTE...  
   URL dopo submit riuscito: / ← corretto  
   Dialog apparso durante il submit? no (corretto)
```

UN DETTAGLIO DI SCRIPTING, NON DELL'APP: PERCHÉ SERVE "FIRE AND FORGET"

Un `window.confirm()` nativo **blocca l'intero thread del renderer** finché non riceve risposta. Il primo tentativo di script di verifica restava appeso per sempre perché aspettava (`await`) la risposta della chiamata CDP che simulava il click — risposta che non sarebbe mai arrivata perché quella stessa chiamata era bloccata dentro il dialog che aveva appena aperto. Fix: inviare il comando di click **senza aspettare la risposta** ("fire and forget"), poi gestire il dialog separatamente.

Ripetuto identico su `shadow-dom-encapsulation`, con l'unica differenza che ogni selettore deve attraversare esplicitamente gli shadow root reali e annidati (`app-root.shadowRoot.querySelector('app-login').shadowRoot...`) — stesso risultato, stesso comportamento corretto in entrambi i casi.

8. Resolver: cosa sono, e perché questo progetto non ne ha bisogno

Un **resolver** pre-carica dati **prima** che il componente della rotta venga creato — la rotta non si attiva finché i dati non sono pronti:

```
export const resolveTask: ResolveFn<Task | undefined> = (route) => {
  const store = inject(TaskStore);
  const id = Number(route.paramMap.get('id'));
  return store.findById(id); // qui sincrono – potrebbe essere http.get(...) reale
};

{ path: 'tasks/:id', component: TaskDetail, resolve: { task: resolveTask } }

// nel componente, grazie a withComponentInputBinding() (già attivo):
readonly task = input<Task>(); // "task" combacia con la chiave del resolve –
                                // stesso meccanismo automatico dei parametri di
rotta
```

PERCHÉ NON È NEL PROGETTO (NOTA ONESTA, NON SOLO UN ESEMPIO)

`TaskDetail` legge già i dati istantaneamente da `TaskStore` (in memoria, sempre disponibile) tramite un `computed()` — non c'è nessuna vera attesa asincrona da "risolvere" prima di mostrare la rotta. Un resolver ha senso quando il dato **non** è già disponibile e richiede una vera chiamata di rete prima di poter mostrare qualcosa di sensato. Aggiungerne uno qui sarebbe un'astrazione ridondante sopra qualcosa che è già istantaneo.

9. Rotte annidate, in breve

Un componente "shell" con il proprio `<router-outlet>` interno, e sotto-rotte che ci si renderizzano dentro:

```
{
  path: 'tasks',
  component: TasksShell, // ha il proprio <router-outlet> nel template
  children: [
    { path: '', component: TaskList },
    { path: ':id', component: TaskDetail },
  ],
}
```

CONFRONTO CON VUE

Identico concetto a Vue Router: `children` + `<router-view>` annidato. Non forzato in questo progetto — la struttura attuale è volutamente piatta (`/` e `/tasks/:id` sono rotte sorelle, non genitore/figlio), ristrutturarla solo per mostrare il concetto sarebbe stato artificioso.

10. Cheat sheet finale

Concetto	API	Nel progetto?
Lazy loading	<code>loadComponent</code>	Sì — tutte le rotte tranne la home
Preloading	<code>withPreloading(PreloadAllModules)</code>	Sì
Guard in ingresso	<code>CanActivateFn</code>	Sì (<code>/new</code>)
Guard in uscita	<code>CanDeactivateFn</code>	Sì (<code>/new</code> , <code>unsaved changes</code>)
Guard su sottoalbero	<code>CanActivateChild</code>	No — richiede rotte annidate
Guard "la rotta esiste?"	<code>CanMatch</code>	No
Dati pre-caricati	<code>ResolveFn</code>	No — dati già in memoria via <code>TaskStore</code>
Rotte annidate	<code>children</code>	No — struttura piatta
Parametri come input	<code>withComponentInputBinding()</code>	Sì (già presente)

11. Appendice: codice sorgente completo

```
src/app/app.routes.ts
```

```

// Le rotte sono un semplice array di oggetti, molto simile a Vue Router.
// Nota tre cose moderne:
// 1) i componenti si importano direttamente e si assegnano a "component"
//    (niente più moduli "feature" con RouterModule.forChild).
// 2) la rotta di creazione task è protetta da un guard FUNZIONALE
//    (canActivateNewTask), non da una classe che implementa CanActivate
//    come si faceva 10 anni fa. È protetta anche in uscita da un guard
//    CanDeactivate (canDeactivateTaskForm) – vedi core/can-deactivate.guard.ts.
// 3) tutte le rotte tranne quella di default usano loadComponent invece
//    di component: il codice di quella feature finisce in un chunk JS
//    SEPARATO, scaricato solo quando l'utente ci naviga davvero (o in
//    anticipo, se è attivo il preloading – vedi app.config.ts). TaskList
//    resta invece "eager" (component diretto): è la home, si carica
//    comunque all'avvio, non ha senso separarla in un chunk a parte.
import { Routes } from '@angular/router';
import { TaskList } from '../features/task-list/task-list';
import { canActivateNewTask } from '../core/auth-guard';
import { canDeactivateTaskForm } from '../core/can-deactivate.guard';

export const routes: Routes = [
  { path: '', component: TaskList, pathMatch: 'full' },
  {
    path: 'login',
    loadComponent: () => import('../features/login/login').then((m) => m.Login),
  },
  {
    path: 'search',
    loadComponent: () => import('../features/task-search/task-search').then((m) =>
m.TaskSearch),
  },
  // withComponentInputBinding (vedi app.config.ts non serve qui perché lo
  // impostiamo tramite provideRouter) fa sì che il parametro di rotta ":id"
  // arrivi direttamente come input() nel componente, senza leggere
  // manualmente ActivatedRoute – vedi TaskDetail.
  {
    path: 'tasks/:id',
    loadComponent: () =>
      import('../features/task-detail/task-detail').then((m) => m.TaskDetail),
  },
  {
    path: 'new',
    loadComponent: () => import('../features/task-form/task-form').then((m) =>
m.TaskForm),
    canActivate: [canActivateNewTask],
    canDeactivate: [canDeactivateTaskForm],
  },
  { path: '**', redirectTo: '' },
];

```

src/app/app.config.ts

```

// app.config.ts raccoglie tutti i "provider" globali dell'applicazione:
// è il posto dove, in Vue, useresti app.use(router) / app.use(pinia) ecc.
// Qui non esiste una classe AppModule: i provider si passano direttamente
// a bootstrapApplication() tramite questo oggetto ApplicationConfig.
import { ApplicationConfig, provideZonelessChangeDetection } from '@angular/core';
import {
  provideRouter,
  withComponentInputBinding,
  withPreloading,
  PreloadAllModules,
} from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    // Zoneless: Angular non usa più zone.js per capire quando ridisegnare
    // la UI. Il change detection è guidato dai signal che leggi nei
    // template – concettualmente più vicino a come già ragioni in Vue
    // (reattività fine-grained, non un ciclo globale "controlla tutto").
    provideZonelessChangeDetection(),

    // Router e HttpClient si registrano come provider "flat", non più
    // via un modulo importato: stessa idea di app.use(router) in Vue,
    // ma passata come funzione a questo array.
    // withComponentInputBinding() fa arrivare i parametri di rotta (:id)
    // direttamente come input() nel componente – vedi TaskDetail.
    // È opt-in, non il default: senza questa riga TaskDetail non
    // riceverebbe mai "id" e servirebbe tornare al vecchio
    // inject(ActivatedRoute) + route.snapshot.paramMap.get('id').
    // Equivalente Vue Router: { path: '/tasks/:id', props: true } nella
    // config della rotta – anche lì va abilitato esplicitamente.
    //
    // withPreloading(PreloadAllModules): compromesso tra "bundle
    // iniziale piccolo" (grazie a loadComponent, vedi app.routes.ts) e
    // "navigazione istantanea" (senza preloading, il PRIMO click su una
    // rotta lazy dovrebbe comunque aspettare il download del chunk).
    // Con questa strategia, Angular scarica in background TUTTI i chunk
    // lazy subito dopo il caricamento iniziale – non li usa ancora, li
    // pre-scarica e basta, così quando l'utente clicca il chunk è già
    // in cache del browser. PreloadAllModules è la strategia built-in
    // più semplice; se ne può scrivere una custom (implementando
    // PreloadingStrategy) per essere più selettivi, es. precaricare
    // solo le rotte marcate con { data: { preload: true } }.
    provideRouter(routes, withComponentInputBinding(),
    withPreloading(PreloadAllModules)),
    provideHttpClient(),
  ],
};

```

src/app/core/can-deactivate.guard.ts

```
// -----
// CanDeactivate: l'opposto di CanActivate – non decide se puoi ENTRARE
// in una rotta, ma se puoi USCIRNE. Caso d'uso classico: un form con
// modifiche non salvate, per evitare che l'utente perda dati per un
// click su un link di navigazione.
// -----
// Come CanActivateFn (vedi auth-guard.ts), è una funzione, non una
// classe – stessa filosofia "guard funzionale" del 2026. La differenza:
// CanDeactivateFn riceve come primo argomento l'ISTANZA del componente
// che si sta per abbandonare, così può interrogarlo direttamente
// invece di dover reggere lo stato altrove (es. in un service).
import { CanDeactivateFn } from '@angular/router';
import { TaskForm } from '../features/task-form/task-form';

export const canDeactivateTaskForm: CanDeactivateFn<TaskForm> = (component) => {
  if (!component.hasUnsavedChanges()) {
    return true;
  }
  // window.confirm() è volutamente il modo più semplice/riconoscibile
  // per dimostrare il pattern – un'app reale userebbe più probabilmente
  // un proprio modal (coerente con lo stile grafico), ma la logica del
  // guard sarebbe identica: true = via libera, false = blocca la
  // navigazione e l'utente resta sulla rotta corrente.
  return window.confirm(
    'Hai scritto un titolo che non hai ancora salvato. Vuoi davvero uscire senza
    creare il task?',
  );
};
```